

SQL Interview

1. Top N Problems

- Top N Records

- Query the 5th largest value in the table t.
- Assume the values are unique, and there are more than 5 values in the table.

```
-- Select values from table t and sort values in descending order.
-- Sort the numbers, use OFFSET , and LIMIT to return the 5th row.
SELECT value
FROM t
ORDER BY value DESC
LIMIT 1
OFFSET 4;
```

- Top N Per Category

Example: Query the 5 highest-rated restaurants in each city.

- highest-rated refers to the highest **average** rating.
- If two restaurants have the same average ratings, return either restaurant.

Table rating:

I.D.	Name	City	Rating
10010	Kim's Kitchen	New York	4
10011	Super Dragon	San Francisco	3
...	...	...	...
12010	Tom's Seafood	Tokyo	2

value	ROW_NUMBER	DENSE_RANK	RANK
5	1	1	1
4.9	2	2	2
4.9	3	2	2
4.8	4	3	4

```
WITH avg_ratings AS (
SELECT
```

```

name, city,
AVG(rating * 1.0) AS avg_rating
FROM rating
GROUP BY name, city
),

rating_rank AS (
SELECT
name, city, avg_rating,
DENSE_RANK() OVER(PARTITION BY city ORDER BY avg_rating DESC) as rk
FROM avg_ratings
)

SELECT
name, city, avg_rating
FROM rating_rank
WHERE rk <= 5;

```

2. Ratios

Example: Subscription Rate

user_id	premium
1	TRUE
2	FALSE
3	TRUE

- Write a query to calculate the **premium subscription rate**: the count of premium subscribers over the total number of users.

```

SELECT
COUNT(user_id) * 1.0 / (SELECT COUNT(user_id) FROM subscription) AS ratio
FROM subscription
WHERE premium = 'TRUE';

SELECT
SUM(CASE WHEN premium = 'TRUE' THEN 1 ELSE 0 END) * 1.0 / COUNT(user_id) AS ratio
FROM subscription;

-- AVG (Case When...Then...Else... End as) -- Best
SELECT
AVG(CASE WHEN premium = 'TRUE' THEN 1.0 ELSE 0.0 END)
AS ratio

```

```
FROM subscription;
```

Example: Immediate Order ([1174. Immediate Food Delivery II](#))

```
WITH ordered_delivery
AS (SELECT
*,
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_rk
FROM delivery)

SELECT
AVG(CASE
WHEN order_date = pref_delivery_date THEN 1.0
ELSE 0.0 END
) AS immediate_percentage
FROM ordered_delivery
WHERE order_rk = 1 # first order
```

3. SQL Categories

Of course, the best way to make sure that you have these 3 categories down is to practice. This post has a list of practice SQL problems to help you do just that. They are broken down into the three categories discussed in the video.

Computing Ratios

- [LeetCode 1174 Immediate Food Delivery II](#)
- [LeetCode 1322 Ads Performance](#)
- [LeetCode 1211 Queries Quality and Percentage](#)

Data Categorization

Categorization

1st step:

- Use CASE WHEN to transform numerical data → categorical data



2nd step:

- Apply aggregation function to obtain the statistics

Data Interview

- [LeetCode 1393. Capital Gain Loss](#)
- [LeetCode 1907. Count Salary Categories](#)
- [LeetCode 1468. Calculate Salaries](#)
- [LeetCode 1212. Team Scores in Football Tournament](#)

Cumulative Sums

```
SELECT
    player_id,
    event_date,
    SUM(games_played)
    OVER (PARTITION BY player_id
          ORDER BY event_date) AS games_played_so_far
FROM Activity;
```

- [LeetCode 1097. Game Play Analysis V](#) (Hard)
- [LeetCode 1454. Active Users](#)
- [LeetCode 579. Find cumulative salary of an employee](#) (Hard)

For even more practice, the following problems will help you practice the self join which you can use to solve cumulative sum problems.

Self Join

- [LeetCode 1270 All People Report to the Given Manager](#)
- [LeetCode 1811 Find Interview Candidates](#)
- [LeetCode 1285 Find the Start and End Number of Continuous Ranges](#)

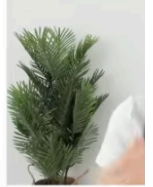
4. Windows Function

Aggregate Function: Sum, Avg, Count, Min, Max

```
SELECT value
      ,SUM(value) OVER(ORDER BY value) AS Total
      ,AVG(value) OVER(ORDER BY value) AS "Avg"
      ,COUNT(value) OVER(ORDER BY value) AS "Count"
      ,MIN(value) OVER(ORDER BY value) AS "Min"
      ,MAX(value) OVER(ORDER BY value) AS "Max"
FROM table
```

value	Total	Avg	Count	Min	Max
3	39	5.57	7	3	8
4	39	5.57	7	3	8
5	39	5.57	7	3	8
5	39	5.57	7	3	8
7	39	5.57	7	3	8
7	39	5.57	7	3	8
8	39	5.57	7	3	8

Find the Team Size



```
SELECT
  employee_id,
  COUNT(employee_id) OVER (PARTITION BY team_id) AS team_size
FROM Employee
```

Employee Table:

employee_id	team_id
1	8
2	8
3	8
4	7
5	9
6	9

Result table:

employee_id	team_size
1	3
2	3
3	3
4	1
5	2
6	2

ata
interview

Ranking Functions: Row_Number, Rank, Dense_Rank

```
SELECT value
  ,ROW_NUMBER(value) OVER(ORDER BY value) AS "ROW_NUMBER"
  ,RANK(value) OVER(ORDER BY value) AS "RANK"
  ,DENSE_RANK(value) OVER(ORDER BY value) AS "DENSE_RANK"
FROM table
```



value	ROW_NUMBER	RANK	DENSE_RANK
3	1	1	1
4	2	2	2
5	3	3	3
5	4	3	3
7	5	5	4
7	6	5	4
8	7	7	5

Ranking Functions



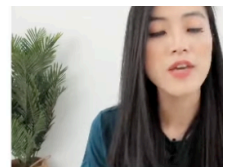
- Step 2: Select employees with rank ≤ 3

```
SELECT
    Department, Employee, Salary
FROM (
    SELECT
        Employee.name AS Employee,
        Employee.Salary AS Salary,
        Department.name AS Department,
        DENSE_RANK() OVER (PARTITION BY Department.id
                           ORDER BY Salary DESC) AS rank
    FROM Employee
    JOIN Department
    ON Employee.departmentId = Department.id
) AS t
WHERE rank <= 3
```

Data
Interview

Analytic Functions: LAG and LEAD

Analytic Functions



- LAG and LEAD
 - Need to specify the column name and offset
 - Offset cannot be a negative value
 - Can set the default value to be used if a previous/following row does not exist

```
LAG (scalar_expression [,offset] [,default])
    OVER ( [ partition_by_clause ] order_by_clause )
```

```
LEAD ( scalar_expression [ ,offset ] , [ default ] )
    OVER ( [ partition_by_clause ] order_by_clause )
```

Data

Analytic Functions

```
SELECT value  
      ,LAG(value, 2) OVER(ORDER BY value) AS "LAG"  
      ,LEAD(value, 2) OVER(ORDER BY value) AS "LEAD"  
FROM table
```

value	LAG	LEAD
3	NULL	5
4	NULL	5
5	3	7
5	4	7
7	5	8
7	5	NULL
8	7	NULL

Define a Window

- OVER clause
 - PARTITION BY - divides the result into partitions
 - ORDER BY - defines the logical order of the rows within each group
 - ROWS/RANGE - specifies the starting and end points of a group

```
OVER (  
    [ <PARTITION BY clause> ]  
    [ <ORDER BY clause> ]  
    [ <ROW or RANGE clause> ]  
)
```

PARTITION BY

User activity

+	-----+	-----+	-----+			
	user_id		date		...	
+	-----+	-----+	-----+			
	1		2021-01-01		...	
+	-----+	-----+	-----+			

```
fun() OVER (PARTITION BY user_id)
```

```
fun() OVER (PARTITION BY user_id, date)
```

ORDER BY

- Defines the logical order of the rows within each partition
- Default order is ascending order

```
<ORDER BY clause> ::=  
ORDER BY order_by_expression  
      [ COLLATE collation_name ]  
      [ ASC | DESC ]  
      [ ,...n ]
```

Data

ROWS/RANGE



- Specify start and end points of a window frame
- Create fixed sized windows

```
<ROW or RANGE clause> ::=  
{ ROWS | RANGE } <window frame extent>  
  
<window frame extent> ::=  
{ <window frame preceding>  
  | <window frame between>  
}  
  
<window frame between> ::=  
BETWEEN <window frame bound> AND <window frame bound>
```

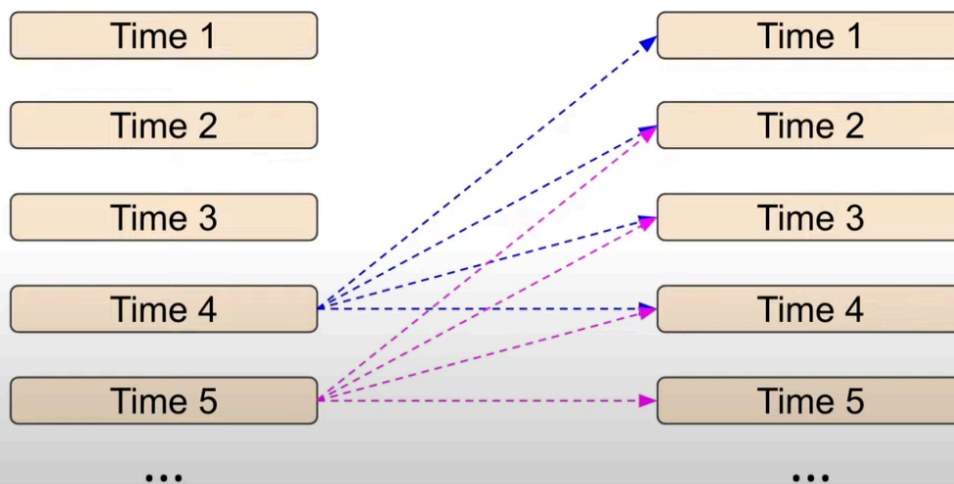
Data

ROWS/RANGE

- ROWS: specify a **fixed number** of rows that precede or follow the current row
- RANGE: specifies the range of values with respect to the **value** of the current row

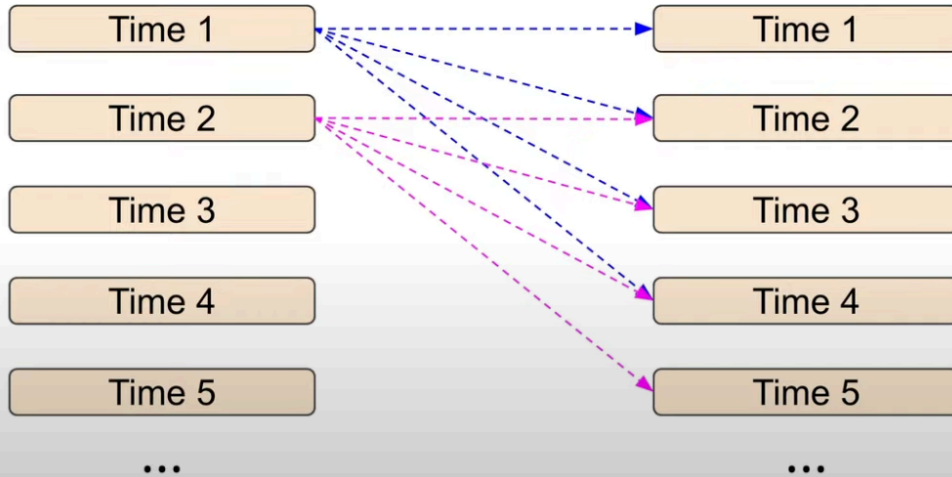
ROWS

```
fun() OVER (ORDER BY date  
            ROWS BETWEEN 3 PRECEDING AND CURRENT ROW)
```



ROWS

```
fun() OVER (ORDER BY date  
            ROWS BETWEEN CURRENT ROW AND 3 FOLLOWING)
```



ROWS/RANGE

- ORDER BY is required

Required
↓

```
fun() OVER (ORDER BY date  
            ROWS BETWEEN 3 PRECEDING AND CURRENT ROW)
```

- Not available for certain functions, eg. ranking functions cannot accept this argument